

# **Logitrack Pro - Documentação Diferenciais**

Por: Eduardo Milhomes Barbosa de Medeiros

## Sumário

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Testes de API.....</b>                           | <b>3</b>  |
| Objetivo.....                                       | 3         |
| Ferramenta e organização.....                       | 4         |
| Endpoints utilizados.....                           | 4         |
| <b>Estrutura das respostas observada.....</b>       | <b>5</b>  |
| Resultados.....                                     | 5         |
| Evidências.....                                     | 6         |
| Observações.....                                    | 7         |
| Cuidados com o ambiente de testes.....              | 7         |
| Como executar.....                                  | 8         |
| <b>Análise de desempenho.....</b>                   | <b>8</b>  |
| Objetivo.....                                       | 8         |
| Método.....                                         | 8         |
| Resultados.....                                     | 9         |
| Resultado por requisição.....                       | 9         |
| Evidências.....                                     | 9         |
| Conclusão e limitações.....                         | 10        |
| Como executar.....                                  | 10        |
| <b>Automação de testes (prova de conceito).....</b> | <b>10</b> |
| Objetivo.....                                       | 10        |
| Ferramenta.....                                     | 11        |
| Organização.....                                    | 11        |
| Testes implementados.....                           | 11        |
| Resultados da execução.....                         | 12        |
| Evidências.....                                     | 12        |
| Decisões de projeto.....                            | 12        |
| Como executar.....                                  | 13        |
| Limitações e próximos passos.....                   | 13        |

# Testes de API

## Objetivo

- Validar diretamente na API (sem passar pela interface) os códigos de resposta, a estrutura das respostas, o controle de acesso e o tratamento de dados inválidos.
- Confirmar, no nível do servidor, os bugs encontrados na execução manual.

## Ferramenta e organização

- **Ferramenta:** Postman, com testes escritos em JavaScript na aba de scripts de cada requisição.
- **Coleção:** automacao/api/LogiTrack-Pro.postman\_collection.json, organizada em pastas (Autenticação, Veículos, Manutenção, Viagens e Dashboard).
- **Variáveis de ambiente:** baseUrl, email e password (configuradas pelo usuário), além de token, tokenInvalido e identificadores de dados de teste, que são preenchidos pelos próprios scripts. O ambiente não faz parte do repositório, para não expor credenciais.
- **Autenticação:** O login retorna um token, usado como Bearer Token nas demais requisições.

## Endpoints utilizados

- Os endereços foram identificados na aba Network do navegador.
- A base da API é api-logitrack.danieldiegosantana.me/v1 (sem https).
- **Autenticação (POST /auth/login):** Login válido e login com senha incorreta.
- **Veículos (GET /veiculos?page=0&size=10):** Listagem.
- **Veículos (POST /veiculos):** Cadastro (inclusive com dado inválido).
- **Veículos (DELETE /veiculos/{id}):** Limpeza dos dados de teste.

- **Manutenção (GET /manutencoes?page=0&size=10):** Listagem.
- **Manutenção (POST /manutencoes):** Cadastro (inclusive com dado inválido).
- **Manutenção (DELETE /manutencoes/{id}):** Limpeza dos dados de teste.
- **Viagens (GET /viagens?page=0&size=10):** Listagem.
- **Viagens (POST /viagens):** Cadastro (inclusive com dado inválido).
- **Viagens (DELETE /viagens/{id}):** Limpeza dos dados de teste.
- **Dashboard (GET /dashboard/total-km):** Total de km da frota.

## Estrutura das respostas observada

- Estrutura registrada a partir das respostas, como base para testes de contrato.
- As listagens e os cadastros retornam os dados dentro do campo data, junto de status, message e timestamp.
- **Login:** status, message, data (userId, name, email, token) e timestamp.
- **Veículos:** id, placa, modelo, tipo (ex.: LEVE) e ano.
- **Manutenção:** id, veiculold, veiculoPlaca, veiculoModelo, dataInicio, dataFinalizacao, tipoServico, custoEstimado e status (ex.: PENDENTE).
- **Viagens:** id, veiculold, dataSaida, dataChegada, origem, destino e kmPercorrida.

## Resultados

- Os testes reprovados falham de propósito: eles descrevem o comportamento esperado e passarão quando os bugs forem corrigidos, podendo ser usados como testes de regressão.
- **Listar veículos, manutenções e viagens:** Validação de Status, formato JSON e tempo de resposta. Resultado esperado: 200, JSON e tempo abaixo do limite. Resultado obtido: 200, JSON e tempo abaixo do limite nas três listagens. (Status: Aprovado)
- **Total de km (Dashboard):** Validação de Status, formato JSON e tempo de resposta. Resultado esperado: 200, JSON e tempo abaixo do limite.

Resultado obtido: 200, JSON e tempo abaixo do limite. (Status: Aprovado)

- **Listar veículos sem autenticação:** Acesso sem token. Resultado esperado: 401 ou 403. Resultado obtido: 403 Forbidden. (Status: Aprovado)
- **Login com senha incorreta:** Código de resposta e emissão de token. Resultado esperado: 401, sem token. Resultado obtido: 200 "Login successful", com token emitido. (Status: Reprovado - BUG-01)
- **Listar veículos com o token emitido por senha incorreta:** Acesso aos dados. Resultado esperado: 401 ou 403. Resultado obtido: 200, com acesso aos dados. (Status: Reprovado - BUG-01)
- **Cadastro de veículo com ano 3000:** Rejeição de dado inválido. Resultado esperado: Erro 4xx. Resultado obtido: 201 Created. (Status: Reprovado - BUG-02)
- **Cadastro de manutenção com datas no ano 3000:** Rejeição de dado inválido. Resultado esperado: Erro 4xx. Resultado obtido: 201 Created. (Status: Reprovado - BUG-03)
- **Cadastro de viagem com chegada anterior à saída:** Rejeição de dado inválido. Resultado esperado: Erro 4xx. Resultado obtido: 201 Created. (Status: Reprovado - BUG-04)
- **Cadastro de viagem com km negativo (-100):** Rejeição de dado inválido. Resultado esperado: Erro 4xx. Resultado obtido: 201 Created. (Status: Reprovado - BUG-05)
- **Total de km antes e depois da viagem com km negativo:** Efeito no indicador. Resultado esperado: Total não alterado por dado inválido. Resultado obtido: Total de 19066.50 antes da exclusão e 19166.50 depois, ou seja, o valor negativo foi subtraído do total. (Status: Reprovado - BUG-05)
- **Cadastro de viagem sobreposta para o mesmo veículo:** Rejeição ou sinalização. Resultado esperado: Erro 4xx. Resultado obtido: 201 Created. (Status: Reprovado - BUG-06)

## Evidências

- As imagens originais encontram-se no diretório de evidências:
- **BUG-01:** Login com senha incorreta e token emitido acessa os dados.
- **BUG-02:** Veículo com ano inválido.
- **BUG-03:** Manutenção com ano inválido.
- **BUG-04:** Viagem com chegada anterior à saída.
- **BUG-05:** Viagem com km negativo e efeito no total (antes e após exclusão).
- **BUG-06:** Viagem sobreposta.

## Observações

- **Controle de acesso:** Sem token, a API negou o acesso aos dados (403), o que é um ponto positivo. O problema está na emissão do token no login (BUG-01).
- **Regra de negócio descoberta:** Ao tentar cadastrar uma manutenção para um veículo que já possuía uma manutenção ativa (Pendente ou Em Realização), a API respondeu 409 com a mensagem de que o veículo já possui manutenção ativa. A regra é correta. Esse resultado não serve como prova do BUG-03, porque a recusa não foi causada pelo ano da data. O teste foi refeito com um veículo de teste sem manutenção ativa, e então a API aceitou a data inválida.
- **Validação de campos obrigatórios:** Uma requisição de manutenção enviada com o corpo vazio retornou 400 com a mensagem "Validation failed" e a lista dos campos obrigatórios. A API valida a presença dos campos, mas, nos casos testados, não validou o conteúdo (ano, km, datas e sobreposição de viagens).
- **Códigos de resposta:** Cadastro retorna 201, exclusão retorna 204 e consulta retorna 200.
- **Limite de tempo de resposta:** O valor de 2000 ms foi adotado por premissa, pois o desafio não define um requisito de desempenho.

## Cuidados com o ambiente de testes

- Cada teste que cria registros foi seguido da exclusão dos dados criados, e todas as exclusões retornaram sucesso. Os dados de teste usaram identificação própria (por exemplo, TESTE-EDUARDO).
- Credenciais e tokens não foram incluídos no repositório. A coleção exportada não contém o ambiente.
- Não foram feitos testes de carga, para não sobrecarregar um ambiente compartilhado.

## Como executar

- Importar a coleção `automacao/api/LogiTrack-Pro.postman_collection.json` no Postman.
- Criar um ambiente com as variáveis `baseUrl` (`api-logitrack.danieldiegosantana.me/v1`), `email` e `password` (credenciais fornecidas no desafio). Marcar a senha como `secret`.
- Executar primeiro a requisição de login válido da pasta Autenticação, que grava o token usado pelas demais.
- Executar as requisições uma a uma. As requisições de cadastro com dado inválido criam registros quando o bug está presente, então cada uma deve ser seguida da requisição de exclusão correspondente (as exclusões usam os identificadores gravados pelos scripts). Não é recomendado executar a coleção inteira de uma só vez pelo Runner, porque os cadastros e as exclusões dependem da ordem.

# Análise de desempenho

## Objetivo

- Realizar uma validação básica do tempo de resposta e do comportamento da API sob múltiplas requisições, conforme proposto no desafio.

## Método

- **Ferramenta:** Collection Runner do Postman, executando a coleção de testes de API descrita acima.
- **Requisições:** Apenas consultas (GET), para não criar nem alterar dados no ambiente compartilhado: Listar veículos, Listar manutenções, Listar viagens, Total de km (Dashboard) e Listar veículos sem autenticação (como contraste).
- **Execução:** 20 iterações, com intervalo de 200 ms entre as requisições. As requisições foram feitas em sequência, e não de forma simultânea.
- **Critério:** Cada listagem foi verificada quanto ao status 200, ao formato JSON e ao tempo de resposta abaixo de 2000 ms. O limite de 2000 ms foi adotado por premissa, pois o desafio não define um requisito de desempenho.
- **Data da execução:** 4 de outubro de 2026.

## Resultados

- **Iterações:** 20
- **Requisições executadas:** 100 (5 requisições em 20 iterações)
- **Duração total:** 51 s 539 ms (inclui o intervalo de 200 ms entre as requisições)
- **Testes executados:** 260
- **Testes aprovados:** 260
- **Testes reprovados:** 0

- **Tempo médio de resposta:** 282 ms

## Resultado por requisição

- **Listar veículos:** 60 aprovados, 0 reprovados
- **Listar veículos sem autenticação:** 20 aprovados, 0 reprovados
- **Listar manutenções:** 60 aprovados, 0 reprovados
- **Listar viagens:** 60 aprovados, 0 reprovados
- **Total de km:** 60 aprovados, 0 reprovados
- Como exemplo de uma das chamadas, a listagem de veículos respondeu com status 200 em 236 ms e 1,3 KB.

## Evidências

- Resultado da execução no Runner do Postman.
- Resumo da execução por requisição.

## Conclusão e limitações

- Nas 80 chamadas de listagem, nenhuma respondeu em mais de 2000 ms, e o tempo médio geral foi de 282 ms. Não foram observadas falhas nem variações de comportamento ao longo das 20 iterações.
- A medição inclui a latência da conexão da máquina utilizada nos testes e foi feita em uma única execução, em um ambiente compartilhado. Os valores servem como referência inicial, e não como garantia de desempenho.
- Esta análise não é um teste de carga, porque as requisições não foram simultâneas. Um teste de carga com volume maior deve ser feito em um ambiente dedicado, para não afetar outros usuários.

## Como executar

- Executar o login válido da pasta Autenticação, para gravar o token.

- Abrir o Runner a partir da coleção e selecionar apenas as requisições de listagem (GET) e a de Total de km.
- Configurar 20 iterações e intervalo de 200 ms, e executar.
- Consultar o tempo médio de resposta e a quantidade de testes aprovados no resumo da execução.

## **Automação de testes (prova de conceito)**

### **Objetivo**

- Automatizar na interface alguns cenários de login executados manualmente.
- Reproduzir, de forma automática, o BUG-01 (senha incorreta aceita), de modo que o teste possa ser reutilizado como teste de regressão depois da correção.

### **Ferramenta**

- Playwright para Python, com o plugin pytest-playwright (Playwright 1.63.0 e pytest-playwright 0.9.0), executando no navegador Chromium.
- As versões exatas das bibliotecas estão em automacao/ui/requirements.txt.
- Os testes foram escritos e executados no Ubuntu, com Python 3.14.4.

### **Organização**

- automacao/ui/tests/test\_smoke.py: Teste de verificação do ambiente: a tela de login carrega com o botão "Entrar".
- automacao/ui/tests/test\_login.py: Testes de login e de acesso ao Dashboard.
- automacao/ui/requirements.txt: Bibliotecas e versões utilizadas.

## Testes implementados

- **test\_tela\_de\_login\_carrega:** Verificação do ambiente.  
Comportamento: O botão "Entrar" aparece na tela de login.  
(Resultado: Aprovado)
- **test\_ct\_log\_01\_login\_valido\_acessa\_o\_dashboard:** Cenário CT-LOG-01.  
Comportamento: Login com credenciais válidas leva ao Dashboard.  
(Resultado: Aprovado)
- **test\_ct\_log\_03\_email\_inexistente\_exibe\_erro:** Cenário CT-LOG-03.  
Comportamento: E-mail inexistente exibe a mensagem de erro e permanece no login. (Resultado: Aprovado)
- **test\_ct\_log\_02\_senha\_incorreta\_deve\_ser\_rejeitada:** Cenário CT-LOG-02. Comportamento: Senha incorreta deve exibir erro e permanecer no login. (Resultado: Falha esperada - BUG-01)
- **test\_dashboard\_sem\_login\_redireciona\_para\_o\_login:** Verificação adicional. Comportamento: Acessar o Dashboard sem estar logado leva ao login. (Resultado: Aprovado)
- O teste do BUG-01 está marcado como falha esperada (xfail), que indica uma falha causada por um defeito conhecido. Assim, a execução normal permanece estável (3 aprovados e 1 falha esperada no arquivo de login), e o defeito continua documentado no próprio teste. Quando o bug for corrigido, o teste passará a ser aprovado e a marcação deve ser removida. Para ver a falha real, o teste pode ser executado com a opção `--runxfail`: nesse caso ele é reprovado, porque o sistema leva o usuário ao Dashboard (/dashboard) e a mensagem de erro não aparece.

## Resultados da execução

- **Execução normal do arquivo de login:** 3 testes aprovados e 1 falha esperada, em 14,31 segundos.
- **Execução com `--runxfail`:** O teste do BUG-01 é reprovado, com a página na URL do Dashboard após o login com senha incorreta.

## Evidências

- Execução normal dos testes de login.
- Execução com `--runxfail`: teste do BUG-01 reprovado.
- Os prints foram feitos quando o arquivo de login tinha três testes. O quarto teste (Dashboard sem login) foi adicionado depois e foi aprovado.

## Decisões de projeto

- **Credenciais fora do código:** O e-mail e a senha são lidos das variáveis de ambiente `LOGITRACK_EMAIL` e `LOGITRACK_PASSWORD`. Os testes que dependem delas são ignorados se as variáveis não estiverem definidas.
- **Sem criação de dados:** Os testes só fazem login e navegação, portanto não deixam registros no ambiente compartilhado.
- **Seletores por papel e nome:** Os campos e botões são localizados por papel e nome acessível (por exemplo, o campo "Email" e o botão "Entrar"), o que torna os testes mais legíveis e menos frágeis. Os nomes foram obtidos com o gerador de código do Playwright (codegen).

## Como executar

Comandos para Linux (Ubuntu), a partir da raiz do repositório:

- `cd automacao/ui`
- `python3 -m venv .venv`
- `source .venv/bin/activate`
- `pip install -r requirements.txt`
- `playwright install chromium`
- `export LOGITRACK_EMAIL="e-mail fornecido no desafio"`
- `read -s -p "Senha: " LOGITRACK_PASSWORD; export LOGITRACK_PASSWORD; echo`
- `pytest -v` (Para o bug real: `pytest -v --runxfail`)

## **Limitações e próximos passos**

- Esta é uma prova de conceito pequena, focada em login e acesso ao Dashboard.
- Os cenários dos demais bugs (BUG-02 a BUG-06) criam dados quando o defeito está presente. Para automatizá-los de forma segura, o ideal é criar os dados pela API e excluí-los ao final de cada teste.
- Os testes podem ser integrados a um pipeline de entrega, para serem executados a cada alteração do sistema.